

PROGRAMACIÓN AVANZADA

Manejo de Archivos –
Orientado a String



Ing. Matias Macrino

MANEJO DE ARCHIVOS

► Clase File:

- Los archivos y directorios del disco, son representados de manera lógica por java a través de un objeto de la clase **File** (dentro de lo que es **java.io.***).
- Dicha clase no solo se utiliza para transferir datos entre la aplicación y el disco, sino para poder obtener información sobre los archivos o directorios, como también la creación y eliminación de los mismos en el disco.

MANEJO DE ARCHIVOS

► Clase File:

- Creación de objeto File: esto mismo se hace instanciando un objeto de dicha clase, el constructor, **File(String path)** permite instancia el objeto a partir de su dirección absoluta o relativa al directorio actual.

Al crear dicho objeto, no quiere decir que se cree el archivo o directorio, lo cual no se realiza de forma implícita.

- La clase File también proporciona un constructor que permite instanciar un objeto File asociado a un fichero, indicando a través de otro objeto File el directorio donde se encuentra el fichero.

File(File dir, String nombre_archivo)

MANEJO DE ARCHIVOS

- ▶ Clase File:

- ▶ Métodos:

- ▶ **boolean createNewFile()**, crea el archivo cuyo nombre ha sido especificado en su constructor. El mismo puede provocar una excepción (**IOException**) que habrá que capturar.
- ▶ **boolean mkdir()**, crea el directorio cuyo nombre ha sido especificado en el constructor.
- ▶ **boolean canRead()**, indica si se puede leer o no del archivo.
- ▶ **boolean canWrite()**, indica si se puede escribir o no en el archivo.
- ▶ **boolean exists()**, indica si existe o no el archivo o directorio indicado en el path.

MANEJO DE ARCHIVOS

- ▶ Clase File:

- ▶ Métodos:

- ▶ ***boolean isFile()***, indica si el objeto creado hace referencia a un archivo o no.
- ▶ ***boolean isDirectory()***, indica si el objeto creado hace referencia a un directorio.
- ▶ ***String getName()***, retorna en una variable String el nombre del archivo sin importar el path.
- ▶ ***String getAbsolutePath()***, retorna en una variable String el path absoluto completo.
- ▶ ***boolean delete()***, elimina el fichero o directorio especificado en el constructor del objeto.
- ▶ ***boolean renameTo(File nuevo_file)***, renombra el archivo asignándole el nombre del objeto File especificado.

MANEJO DE ARCHIVOS

- ▶ Escritura de un archivo de texto:
 - ▶ Creación de un objeto `FileWriter`. Para poder iniciar la escritura en un archivo es necesario un objeto **`FileWriter`**, el cual nos permite tener acceso al archivo en modo “escritura”. Posee dos constructores:
`FileWriter(String path, boolean append)`
`FileWriter(File fichero, boolean append)`

Donde el parámetro **`append`** permite indicar si los datos que se escribirán se añadirán a los existentes (true) o sobrescribir estos (false)

MANEJO DE ARCHIVOS

- ▶ BufferedWriter:

- ▶ Es una subclase de la clase abstracta `Writer`. La clase `BufferedWriter` permite el uso del bufer para los flujos basados en caracteres.
- ▶ Para instanciar un objeto de dicha clase, lo podemos hacer con el siguiente constructor.
`BufferedWriter(Writer obj)`
- ▶ Para poder escribir en un archivo poseemos el método **`write(String str)`**

MANEJO DE ARCHIVOS

- ▶ Lectura de un archivo:
- ▶ Muchas de las operaciones realizadas con ficheros en un programa tiene que ver con la lectura de archivos de texto.
- ▶ Para recuperar cadenas de caracteres de un archivo, el paquete **java.io** nos proporciona dos clases **FileReader** y **BufferedReader**

MANEJO DE ARCHIVOS

- ▶ Lectura de un archivo:

- ▶ Creación de un objeto FileReader:

- ▶ Para recuperar información de archivo de texto, es necesario primeramente instanciar un objeto **FileReader asociado a un objeto File**.
- ▶ Un objeto **FileReader** representa un archivo de texto “abierto” para la lectura de datos.
- ▶ Es capaz de adaptar la información recuperada del archivo, transformando los bytes almacenados en caracteres Unicode.

MANEJO DE ARCHIVOS

► Lectura de un archivo - FileReader:

- Se puede construir un objeto FileReader a partir de un objeto File existente o bien proporcionando la ruta del archivo.
 - `FileReader(String path) → FileReader fr = new FileReader("datos.txt");`
 - `FileReader(File Object) → FileReader fr = new FileReader(file);`
- Nos proporciona el método **`read()`** para la lectura de la información almacenada en un archivo.
- Recupera dicha información en formato de byte, debiendo ser convertido posteriormente a ***String***.

MANEJO DE ARCHIVOS

► Lectura de un archivo - *BufferedReader*:

- Para evitarnos realizar el casteo ya mencionado, utilizaremos un objeto del tipo ***BufferedReader***.
- Para poder instanciar este objeto lo debemos hacer a partir de un objeto ***FileReader*** ya creado.
 - ***BufferedReader bf = new BufferedReader(fr);***
- A través del método ***readLine()*** podemos leer líneas de texto del archivo. Para recuperar todas las líneas de texto, debemos de crear un bucle.

MANEJO DE ARCHIVOS

► Lectura de un archivo - *BufferedReader*:

- El método ***readLine()*** apunta a la siguiente línea de texto, luego de recuperar la actual. Al no existir mas, retorna ***null***.
- Si deseamos leer carácter a carácter, se debe de cambiar el método por ***read()***.
- ***read()*** retorna un entero que representa el código Unicode del carácter, en el caso de no encontrar mas caracteres retorna ***-1***.

MANEJO DE ARCHIVOS

- ▶ Lectura de un archivo - Scanner:
- ▶ Para realizar la lectura de un archivo, se puede hacer a través del objeto Scanner.
- ▶ Creación de un objeto Scanner. Para poder instanciar un objeto Scanner lo podemos hacer con el siguiente constructor **Scanner(File obj_file)**.
- ▶ Además de hacer mas sencilla la tarea de lectura de datos de usuario a través de teclado, la utilización de dicha clase permite simplificar el proceso de recuperación de datos dentro de un archivo.

MANEJO DE ARCHIVOS

- ▶ Métodos de la clase Scanner:
- ▶ ***String next()***. Devuelve el siguiente token
- ▶ ***boolean hasNext()***. Indica si existe un nuevo token para leer o no.
- ▶ ***aaa nextAaa()***. Devuelve el siguiente token, como una variable nativa, siendo aaa el tipo de variable.
- ▶ ***boolean hasNextAaa()***. Indica si existe o no un siguiente token del tipo especificado, siendo Aaa el nombre del tipo.

MANEJO DE ARCHIVOS

► Archivos – Ejercitación :

1. Escribir el siguiente String “Mi primer uso de archivos”, dentro de un archivo con el nombre “Mi primer archivo”. En caso de no existir el mismo, debe ser generado.
2. Leer archivo con el nombre “Mi primer archivo” y mostrar por pantalla lo que contenga escrito.
3. Generar un archivo con el siguiente nombre “Archivo en carpeta” y guardarlo dentro de la carpeta “Archivos Generales”. Por ultimo reemplazar el nombre de “Archivo en carpeta” por “Archivo guardado en carpeta”.
4. Crear un objeto persona con los siguientes datos: nombre, apellido, fecha de nacimiento, dni y dirección.
5. Almacenar los datos del objeto persona en un archivo, tener en cuenta que debe de poder almacenar mas de un objeto persona dentro del archivo.
6. Recuperar los datos de este ultimo archivo generado y obtener los datos de las personas en pantalla.

PROGRAMACIÓN AVANZADA

Manejo de Archivos
Orientado a bytes



Ing. Matias Macrino

MANEJO DE ARCHIVOS

- ▶ Escritura de datos primitivos:
- ▶ Las clases **FileWriter** y **FileReader** están pensadas para escribir y recuperar, respectivamente, cadenas de caracteres en un archivo.
- ▶ Para poder almacenar datos en un archivo en forma de tipos nativos de Java, el paquete **java.io** proporciona las clases **FileOutputStream** y **DataOutputStream**

Tipo	Corrientes de bytes	Corrientes de Caracteres
Archivos	FileInputStream	FileReader
	FileOutputStream	FileWriter

MANEJO DE ARCHIVOS

- ▶ FileOutputStream:

- ▶ Dicha clase fue diseñada para escribir datos a un objeto del tipo **File**.
- ▶ La intención de esta clase es escribir corrientes (Stream) de bytes en bruto en un archivo como pueden ser, por ejemplo, archivos de imagen.
- ▶ Este objeto es el equivalente al **FileWriter** utilizado en la escritura de cadenas de caracteres. Como en el caso de éste, se puede crear un objeto **FileOutputStream** que permita añadir información al fichero o sobrescribirla haciendo uso de los constructores.

FileOutputStream (File fichero, boolean append)

FileOutputStream (String path, boolean append)

MANEJO DE ARCHIVOS

- ▶ DataOutputStream:

- ▶ A partir del objeto **FileOutputStream** se puede crear un objeto **DataOutputStream** para realizar la escritura de los datos.

El constructor a utilizar sería:

```
DataOutputStream(OutputStream)
```

- ▶ Dicha clase proporciona métodos para escribir datos en un fichero en cada uno de los tipos nativos de Java. Estos métodos tienen el formato:

```
void writeXxx(Xxx dato)
```

Siendo Xxx el nombre del tipo primitivo de Java.

MANEJO DE ARCHIVOS

- ▶ Lectura de datos primitivos:
- ▶ Para realizar la lectura de los datos almacenados en un archivo mediante ***DataOutputStream***, el paquete ***java.io*** proporciona la clase ***DataInputStream*** que a su vez se apoya en la clase ***FileInputStream***.

MANEJO DE ARCHIVOS

- ▶ *FileInputStream*:
- ▶ Esta clase se utiliza para leer bytes desde un archivo.
- ▶ Se diseñó con la intención de leer bytes en bruto desde una corriente (***Stream***), como por ejemplo puede ser, una serie de bytes pertenecientes a una imagen.
- ▶ Es el equivalente a la clase ***FileReader*** utilizada para la lectura de cadenas de caracteres. Se puede crear un objeto ***FileInputStream*** a partir de sus constructores.
 - FileInputStream(File archivo)
 - FileInputStream(String path)

MANEJO DE ARCHIVOS

- ▶ DataInputStream:

- ▶ A partir del objeto **FileInputStream** se puede crear un objeto **DataInputStream** para realizar la lectura de los datos.

El constructor a utilizar sería:

`DataInputStream(InputStream)`

- ▶ Dicha clase proporciona métodos para la lectura de datos en un fichero en cada uno de los tipos nativos de Java. Estos métodos tienen el formato:

`Xxx readXxx(Xxx dato)`

Siendo Xxx el nombre del tipo primitivo de Java.

MANEJO DE ARCHIVOS

► Ejemplo de escritura:

```
import java.io.DataOutputStream;

public class Escritura {
    public static void main(String[] args) {
        try {
            FileOutputStream fOut = new FileOutputStream(new File("datos.dat"));
            DataOutputStream dOut = new DataOutputStream(fOut);
            //array de entero
            int[] arr = {5,10,3,6};
            for(int i = 0; i < arr.length; i++) {
                dOut.writeInt(arr[i]);
            }
            dOut.close();
        } catch(IOException e) {
            System.err.println("Ocurrio un error: " + e.getMessage());
        }
    }
}
```

MANEJO DE ARCHIVOS

► Ejemplo de lectura:

```
import java.io.DataInputStream;

public class Lectura {
    public static void main(String[] args) {
        try {
            DataInputStream dIn = new DataInputStream(new FileInputStream(new File("datos.dat")));
            //bucle infinito
            for(;;){
                System.out.println(dIn.readInt());
            }
        } catch(IOException e) {
            System.err.println("Ocurrio un error: " + e.getMessage());
        }
    }
}
```

- Cada método **read** devuelve el tipo de dato actual y tras la lectura de este se posiciona en el siguiente dato del archivo. Cuando se llegue al final del archivo se producirá una excepción **EOFException** al invocar al método **read**.

MANEJO DE ARCHIVOS

► Ejemplo de lectura:

```
import java.io.DataInputStream;

public class Lectura {
    public static void main(String[] args) {
        try {
            DataInputStream dIn = new DataInputStream(new FileInputStream(new File("datos.dat")));
            //bucle infinito
            for(;;){
                System.out.println(dIn.readInt());
            }
        } catch (IOException e) {
            e.printStackTrace();
            System.err.println("Ocurrio un error: " + e.getMessage());
        }
    }
}
```

Problems @ Javadoc Declaration Console

<terminated> Lectura [Java Application] C:\Program Files (x86)\Java\jre8\bin\ja

5
10
3
6
java.io.EOFException
at java.io.DataInputStream.readInt(Unknown Source)
at Lectura.main(Lectura.java:12)
Ocurrio un error: null

MANEJO DE ARCHIVOS

- ▶ Escritura de objetos en un fichero:
- ▶ Además de poder escribir en un archivo cadena de caracteres y tipos de datos primitivos de Java. Se puede intercambiar otros tipos de objetos Java entre una aplicación y el disco.
- ▶ Para poder almacenar un objeto en disco, el mismo si o si debe de ser **Serializable**.

MANEJO DE ARCHIVOS

- ▶ Serialización:
- ▶ Solo son serializados los datos de los objetos (variables de instancia).
- ▶ Con la palabra clave ***transient***, la variable de instancia no será serializada.
- ▶ Al almacenar una corriente (stream), acción conocida como persistencia, solo se preservan los datos, métodos y constructores no son parte de la corriente de serialización.
- ▶ Los datos son escritos en la corriente con un formato de composición de bytes y cadenas representadas como caracteres UTF
- ▶ La interfaz Serializable pertenece al paquete java.io

MANEJO DE ARCHIVOS

```
import java.io.Serializable  
public class Persona implements Serializable {  
    private String name;  
    private int edad;  
    public Persona() { }  
    public void setName(String n) { name = n; }  
    public String getName() { return name; }  
    public void setEdad(int e) { edad = e; }  
    public int getEdad() { return edad; }  
}
```

MANEJO DE ARCHIVOS

- ▶ ObjectOutputStream:

- ▶ La escritura de objetos se realiza a través de **ObjectOutputStream**.
- ▶ Para la creación de un objeto de esta clase es necesario disponer de **FileOutputStream** asociado al archivo donde se va a escribir.
- ▶ La escritura del objeto se realiza con el siguiente método **writeObject(Object obj)**

MANEJO DE ARCHIVOS

- ▶ ObjectInputStream:

- ▶ La lectura de objetos se realiza a través de **ObjectInputStream**.
- ▶ Para la creación de un objeto de esta clase es necesario disponer de **FileOutputStream** asociado al archivo donde se va a escribir.
- ▶ La lectura del objeto se realiza con el siguiente método **readObject()**.
Al llamar a dicho método, se produce la **deserializacion** del objeto que básicamente consiste en la reconstrucción de este a partir de la información recuperada.
Los datos no serializables, serán inicializados utilizando el constructor por defecto de la clase en cuestión.

MANEJO DE ARCHIVOS

```
public class ObjetoSerializado implements Serializable{

    private String cadena;
    private int numero;
    private transient String cadenaNoSerializable;

    public ObjetoSerializado() { }

    public String getCadena() {
        return cadena;
    }

    public void setCadena(String cadena) {
        this.cadena = cadena;
    }

    public int getNumero() {
        return numero;
    }

    public void setNumero(int numero) {
        this.numero = numero;
    }

    public String getCadenaNoSerializable() {
        return cadenaNoSerializable;
    }

    public void setCadenaNoSerializable(String cadenaNoSerializable) {
        this.cadenaNoSerializable = cadenaNoSerializable;
    }

}
```

MANEJO DE ARCHIVOS

```
public class Escritura {  
  
    public static void main(String[] args) {  
  
        ObjetoSerializado obj = new ObjetoSerializado();  
  
        try {  
            obj.setCadena("Cadena main");  
            obj.setNumero(100);  
            obj.setCadenaNoSerializable("Cadena no serializada main");  
            FileOutputStream fOut = new FileOutputStream(new File("objeto.dat"));  
            ObjectOutputStream oOut = new ObjectOutputStream(fOut);  
            oOut.writeObject(obj);  
            oOut.close();  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
  
    }  
  
}
```

MANEJO DE ARCHIVOS

```
public class Lectura {  
  
    public static void main(String[] args) {  
        ObjetoSerializado obj = new ObjetoSerializado();  
  
        try {  
            FileInputStream fIn = new FileInputStream(new File("objeto.dat"));  
            ObjectInputStream oIn = new ObjectInputStream(fIn);  
            obj = (ObjetoSerializado) oIn.readObject();  
            System.out.println("Cadena: " + obj.getCadena() + "\tNumero: " + obj.getNumero() +  
                               "\tcadenaNoSerializable: " + obj.getCadenaNoSerializable());  
            oIn.close();  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Problems @ Javadoc Declaration Console

<terminated> Lectura (1) [Java Application] C:\Program Files (x86)\Java\jre8\bin\javaw.exe (12/4/2017 10:21:30)

Cadena: Cadena main Numero: 100 cadenaNoSerializable: null